NutriMaker con MySQL

Documento de Diseño y Arquitectura de Software Actualizado

Resumen Ejecutivo del Cambio de Arquitectura:

El sistema NutriMaker ha sido migrado de una arquitectura basada en llamadas HTTP REST a Oracle Cloud (ORDS) hacia una arquitectura cliente de escritorio pesada con conexión directa a MySQL mediante JDBC (MySQL Connector/J 9.7.0). Se sustituyeron los repositorios REST por consultas SQL parametrizadas, se incorporaron transacciones ACID para creación y clonación de dietas, se filtraron las consultas por el usuario autenticado y se gestionan las credenciales de base de datos mediante variables de entorno.

1. Introducción y Tecnologías

1.1 Propósito y Alcance: Este documento actualiza la arquitectura y diseño detallado de NutriMaker para reflejar la sustitución de la capa de persistencia remota sobre Oracle REST Data Services (ORDS) por un esquema relacional MySQL local/servidor accedido directamente vía JDBC. La aplicación conserva su propósito de permitir a un nutriólogo registrar pacientes, diseñar, generar, clonar, consultar, eliminar y exportar a PDF dietas personalizadas.

1.2 Stack Tecnológico Actualizado:

Lenguaje y GUI	Java 23 / JDK 25 con módulos (module-info.java) y JavaFX 23.0.1 para la interfaz de usuario.
Acceso a Datos	JDBC nativo con MySQL Connector/J 9.7.0 y consultas SQL parametrizadas.
Motor de Base de Datos	MySQL Server 8.0.16+ (o MariaDB 12.3.2) con esquema relacional en database/mysql/01_schema.sql.
Seguridad	Hashes BCrypt para almacenamiento de contraseñas de usuario (jBCrypt). Credenciales mediante variables de entorno.
Exportación a PDF	iText 7 (kernel, layout, io) con tipografía incrustada Comic Shanns Mono Nerd Font.
Construcción	Apache Maven administrado con Wrapper (mvnw.cmd).

2. Cambios Arquitectónicos y Estilo de Persistencia

En la arquitectura previa, la aplicación cliente enviaba solicitudes HTTP sincrónicas mediante OkHttp y Gson a endpoints de Oracle REST Data Services (ORDS). La nueva arquitectura elimina la dependencia de la API REST y conecta los repositorios JavaFX directamente al servidor MySQL mediante JDBC.

- **Sustitución de llamadas REST por JDBC:** UserRepository, PatientRepository, MealRepository y DietRepository ejecutan directamente PreparedStatements con SQL parametrizado, eliminando la clase DatabaseClient.
- **Esquema Relacional Normalizado:** Se definió el archivo database/mysql/01_schema.sql con 7 tablas relacionables mediante claves foráneas, índices e identificadores AUTO_INCREMENT.
- **Manejo de Transacciones ACID:** Operaciones compuestas como la creación de dietas y la clonación de dietas utilizan transacciones explícitas (connection.setAutoCommit(false)). Si alguna inserción falla, se ejecuta rollback de forma inmediata.
- **Obtención Segura de Identificadores:** El ID del paciente y de la dieta recién insertados se recuperan mediante las llaves devueltas por la propia inserción (getGeneratedKeys()), erradicando la vulnerabilidad de consultar el 'último registro global' de la base de datos.
- **Aislamiento por Usuario Autenticado:** El conteo global de dietas y el listado paginado se filtran estrictamente por el user_id del usuario que inició sesión.
- **Integridad de Comidas Programadas:** Al recuperar una dieta, las comidas en diet_meal se agrupan por cada registro programado (día, hora y tipo), evitando que alimentos repetidos en distintos días se eliminen entre sí.

3. Modelo de Datos Relacional en MySQL

El esquema relacional de NutriMaker está contenido en database/mysql/01_schema.sql y consta de 7 tablas diseñadas para mantener la integridad referencial y soportar el catálogo nutricional y las dietas.

Tabla	Contenido y Propósito	Llaves e Índices
useraccount	Cuentas de usuario de la aplicación. Guarda correo electrónico y hash BCrypt.	PK: user_id (AUTO_INCREMENT) UNIQUE: email
patient	Información física de pacientes: nombre, edad, peso y estatura.	PK: patient_id (AUTO_INCREMENT)
mealbase	Catálogo base de alimentos con valores calóricos y macronutrientes.	PK: meal_base_id (AUTO_INCREMENT)
ingredient	Nombre global de ingredientes de los alimentos.	PK: ingredient_id (AUTO_INCREMENT)
meal_ingredient	Relación N:M entre alimentos e ingredientes con cantidades y unidades.	FK: meal_base_id, ingredient_id
diet	Registro general de la dieta:	PK: diet_id (AUTO_INCREMENT)

	usuario, paciente, totales nutricionales, día de descanso, comidas/día y nota.	FK: user_id, patient_id
diet_meal	Comidas programadas en la dieta: mapea cada alimento con su día, hora y tipo de comida.	FK: diet_id, meal_base_id CASCADE DELETE en diet_id

4. Seguridad y Configuración de Entorno

4.1 Variables de Entorno: Para cumplir con las restricciones de seguridad (RNF-SEG-05), la cadena de conexión JDBC y las credenciales ya no están escritas en el código fuente ni expuestas en diagramas. Se leen dinámicamente de las variables de entorno:

- **NUTRIMAKER_DB_URL:** URL JDBC de conexión. Ejemplo: jdbc:mysql://localhost:3306/nutrimaker?connectionTimeZone=UTC&forceConnectionTimeZoneToSession=true
- **NUTRIMAKER_DB_USER:** Usuario del servidor MySQL con permisos reducidos sobre el esquema nutrimaker (ej. nutrimaker).
- **NUTRIMAKER_DB_PASSWORD:** Contraseña requerida para establecer la conexión a MySQL.

4.2 Protección de Contraseñas: Las cuentas registradas en la ventana de NutriMaker se guardan en la tabla useraccount. Las contraseñas de los usuarios de la aplicación se almacenan de forma irreversible utilizando hashes BCrypt generados por la librería jBCrypt.

5. Verificación, Pruebas y Deuda Técnica Resuelta

Se implementó la suite de pruebas de integración en src/test/java/com/javafx/nutrimaker/repository/MySqlMigrationTest.java. Las pruebas verificaron exitosamente los siguientes puntos contra una instancia de prueba:

- Autenticación y registro con hash BCrypt y consultas SQL parametrizadas.
- Alta de pacientes y recuperación del ID devuelto por AUTO_INCREMENT.
- Generación completa de dieta, almacenamiento de comidas programadas y cálculo de totales.
- Clonación de dieta mediante transacción con rollback automático ante fallas.
- Paginación y conteo de dietas filtrado estrictamente por user_id.
- Agrupación adecuada de comidas repetidas y borrado en cascada (CASCADE DELETE) al eliminar una dieta.

Resolución de Riesgos y Deuda Técnica de la Versión Previa:

Problema Previo (ORDS REST)	Solución Aplicada en MySQL	Estado
Uso del 'último ID global' en toda la BD provocaba mezcla de datos entre usuarios simultáneos.	Se utilizan llaves devueltas directamente por la inserción SQL (AUTO_INCREMENT / getGeneratedKeys).	**Resuelto**

Crear o clonar dietas no tenía transacción; fallas parciales dejaban datos incompletos.	Manejo explícito de transacciones con commit() y rollback() en las operaciones compuestas.	**Resuelto**
Conteo total de dietas no filtraba por user_id.	Consultas SQL con cláusula WHERE user_id = ? en el conteo y la paginación.	**Resuelto**
Repetir un alimento en distintos días borraba repeticiones al recuperar la dieta.	Agrupación corregida por registro programado (día, hora y tipo) en diet_meal.	**Resuelto**